

Verilog 硬體描述語言 (I)

- 隨機存取記憶體(RAM)
- 唯讀記憶體(ROM)

Random Access Memory (RAM) (1)

- ❑ 電路中需要大量儲存資料的元件為：隨機存取記憶體 (RAM)或是唯讀記憶體 (ROM)。
- ❑ 隨機存取記憶體與唯讀記憶體的差異：
 - 可以將資料寫入到隨機存取記憶體(RAM)中或是從隨機存取記憶體中讀出資料。
 - 我們只能從唯讀記憶體(ROM)中讀出資料，但不能寫入資料。
- ❑ 隨機存取記憶體(RAM)為一種序向電路，它是由一些基本的記憶單元 (Memory Cell, MC) 與位址解碼器 (Address Decoder) 所組成的；而其內的每一個記憶單元均由一個正反器與一些控制此正反器的資料寫入與讀取動作的邏輯閘電路所組成。
- ❑ RAM組成：通常隨機存取記憶體都有以下的控制訊號：/CS (晶片選擇, Chip Select)、/RW (讀取/寫入, 0: Read, 1: Write)、Address (位址線)、Data_In (資料輸入線) 以及 Data_Out (資料輸出線)。

Random Access Memory (RAM) (2)

// RAMW5B8.V : 具有5個字組, 每個字組有8個位元的記憶體模組(RAM)

```
module RAM_Word5_Bit8 (CS, RW, Address, Data_In, Data_Out);
```

```
parameter Words = 5;
```

```
input CS, RW;
```

```
input [2:0] Address;
```

```
input [7:0] Data_In;
```

```
output [7:0] Data_Out;
```

```
reg [7:0] RAM_Data[Words-1:0];
```

```
reg [7:0] Data_Out;
```

```
always @(negedge CS)
```

```
begin // 負緣觸發, 選擇到此晶片
```

```
if (RW == 1'b0) // 0=讀取
```

```
    Data_Out = RAM_Data[Address[2:0]];
```

```
else
```

```
begin
```

```
    if (RW == 1'b1) // 1=寫入
```

```
        RAM_Data[Address[2:0]] = Data_In;
```

```
    else
```

```
        Data_Out = 8'dz; // 高阻抗 (High Impedance)
```

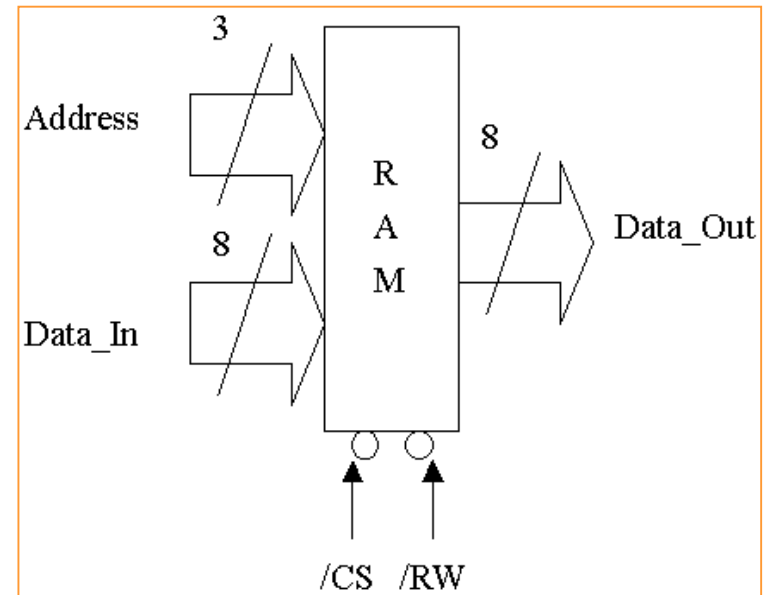
```
end
```

```
end
```

```
endmodule
```

$2^3=8$ word

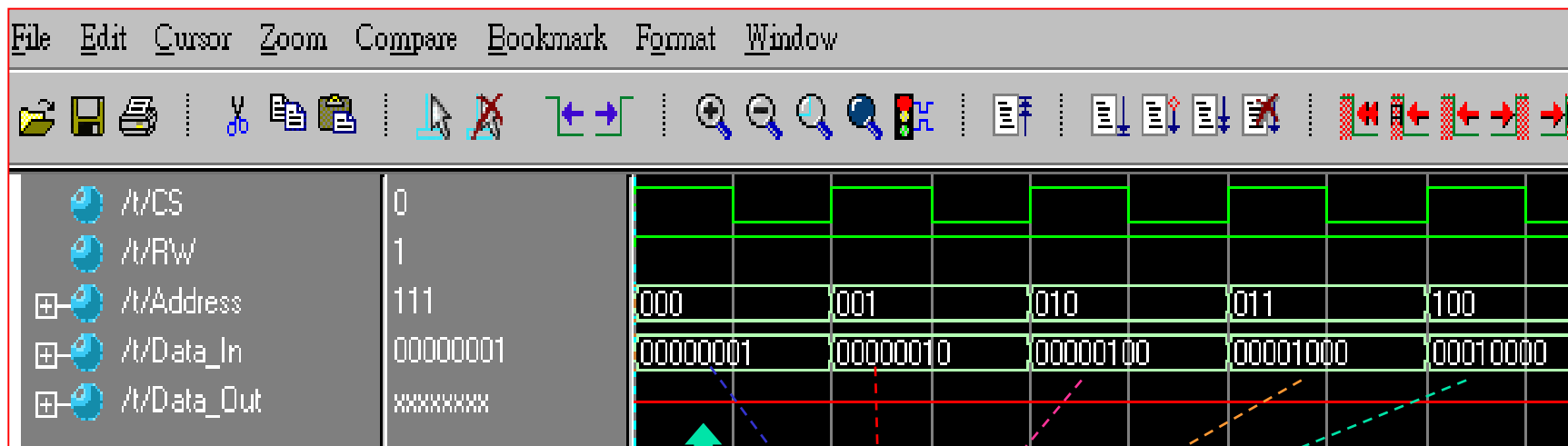
Memory cell



RW=1 $\Rightarrow$ Data into RAM_Data $\Rightarrow$ RW=0 $\Rightarrow$ Data go to Data_out

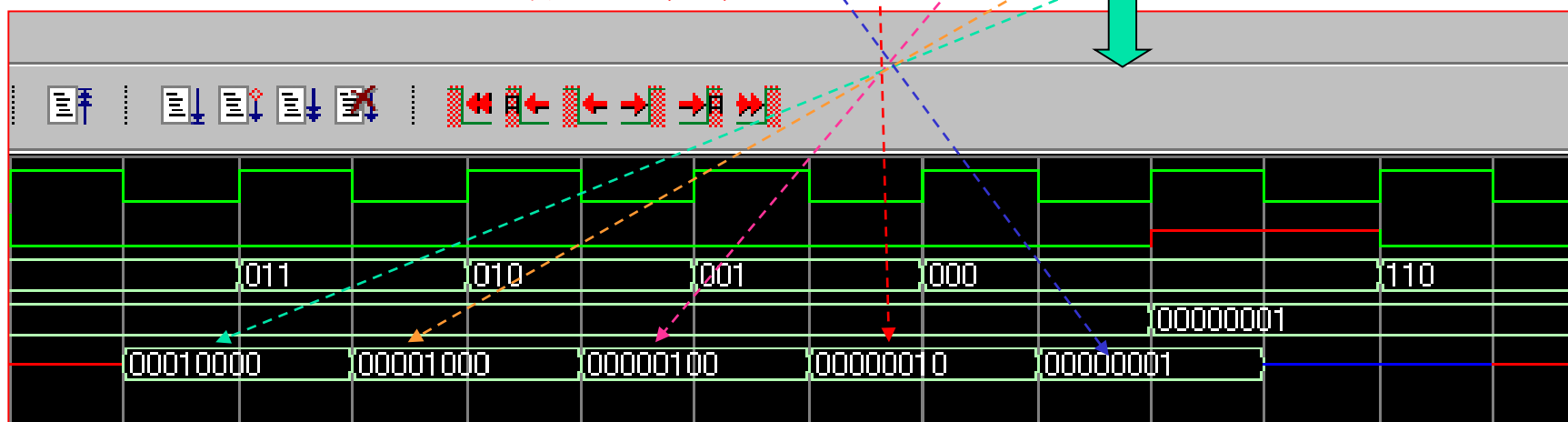
Simulation of RAMW5B8.V

5_03_RAMW5B8



R/W=1
將Data寫入位址2

R/W=0
將位址2的Data讀出



Bits Extension of RAM (1)

- ❑ 隨機存取記憶體(RAM)的擴充方式有：位元組 (Bits) 的擴充及字組 (Words) 的擴充這二種方式。
- ❑ 隨機存取記憶體(RAM)的擴充有一個限制條件：必須是”同樣位元組 (Bits) 大小”及”字組 (Words)大小相同”，而且具有相同控制訊號功能的 RAM，才可以組成隨機存取記憶體的擴充電路。

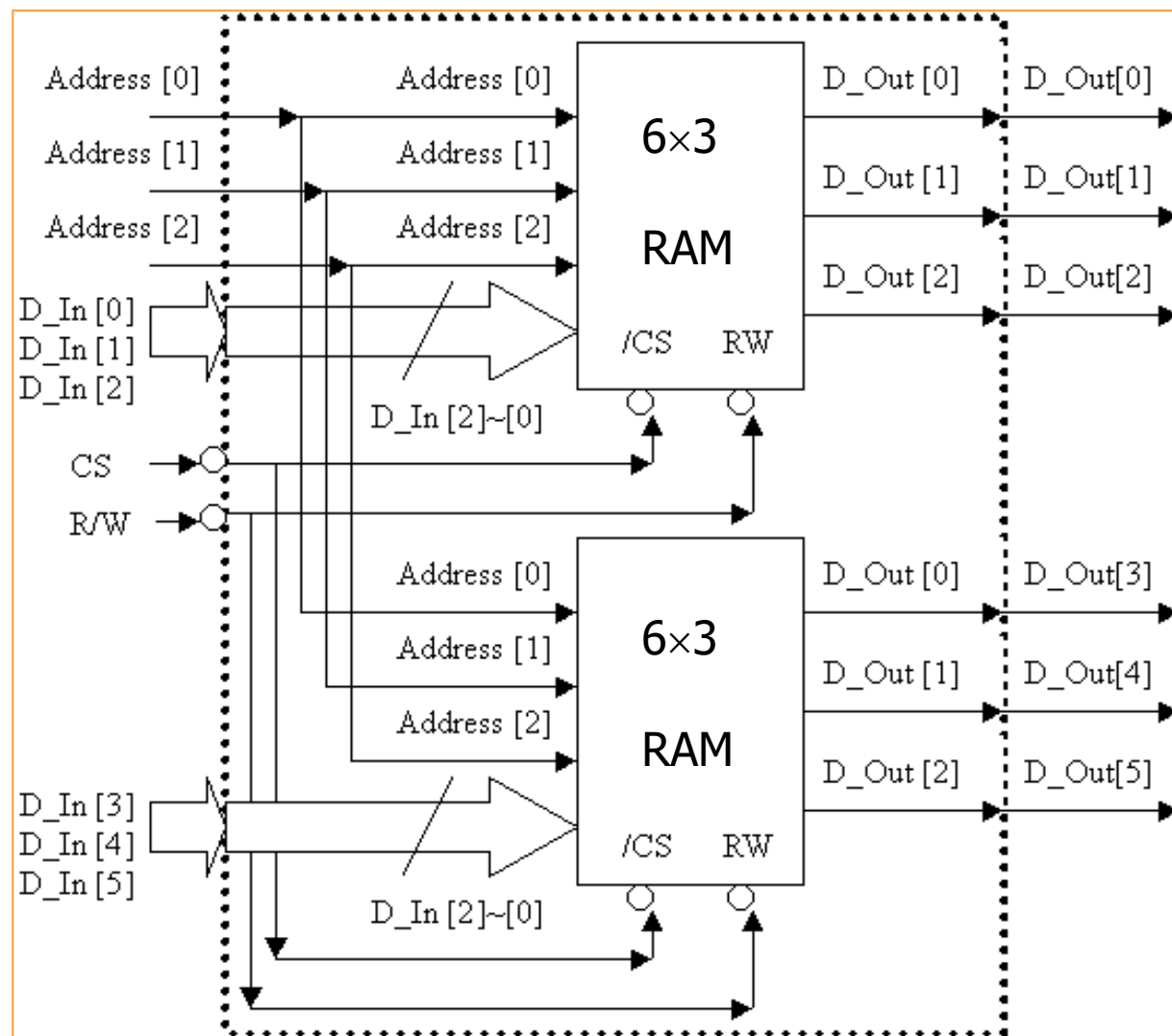
- ❑ **範例：RAM之位元組 (Bits)擴充方式**

我們為了設計一個具有”6個字組”且”每個字組有6個位元”的隨機存取記憶體模組電路 (**6×6 RAM**)，故需要使用兩個具有”6個字組且每個字組有3個位元”的隨機存取記憶體模組 (**6×3 RAM**) **並聯組成**。

- 首先，我們將所有**6×3 RAM**的晶片選擇線 (Chip Select)、讀寫控制線 (Read / Write) 與位址線 (Address) 全部並聯起來當作是**6×6 RAM** 的晶片選擇線 (Chip Select)、讀寫控制線 (Read / Write) 以及位址線 (Address)。
- 再將其中一個**6×3 RAM** 的資料輸入線 (Data In) 接到**6×6 RAM**資料輸入線 (Data In) 的位元 2 到位元 0 (Data_In[2:0])；另一個 **6×3 RAM** 的資料輸入線 (Data In) 接到**6×6 RAM**資料輸入線 (Data In) 的位元 5 到位元 3 (Data_In [5:3])。
- 最後，將其中一個 **6×3 RAM** 的資料輸出線 (Data Out) 接到 **6×6 RAM**資料輸出線 (Data Out) 的位元 2 到位元 0 (Data_Out[2:0])；另一個 **6×3 RAM** 的資料輸出線 (Data Out) 接到 **6×6 RAM**資料輸出線 (Data Out) 的位元 5 到位元 3 (Data_Out[5:3])。
- 我們利用將這兩個 **6×3 RAM**的晶片選擇線 (Chip Select) 並聯起來的目的是這兩個 6×3 RAM均同時被選擇到或者是沒有被選擇到。若同時被選擇且讀寫控制線 (Read / Write) 為0時，則這兩個**6×3 RAM**各輸出3個位元；若同時被選擇且讀寫控制線 (Read / Write) 為1時，則輸入的資料分別寫入到這兩個**6×3 RAM**內。因此，**電路的位元組 (Bits) 數目就結合為6個位元了**，達到了位元組擴充的目的。

Bits Extension of RAM (2)

由二個6×3 RAM組成一個6×6 RAM的邏輯符號圖。



Bits Extension of RAM (3)

□ RAM_6_6.V：位元組 (Bits) 的擴充範例

利用 ``include` 編譯命令設計一個具有”6個字組、每個字組有6個位元”的隨機存取記憶體的 Verilog 電路描述檔 (RAM_6_6.V)，因此我們引用已經設計好的一個具有”6個字組、每個字組有3個位元”的隨機存取記憶體的 Verilog 電路描述檔 (RAM_6_3.V) 來一起編譯合成。

□ 範例：

`// RAM_6_6.V：運用2個具有6個字組, 每個字組有3個位元的 RAM`

`// 來設計出1個具有6個字組, 每個字組有6個位元的 RAM`

`// 引用 RAM_6_3.V，一個具有6個字組, 每個字組有3個位元的 RAM`

``include "RAM_6_3.V"`

`// 具有6個字組, 每個字組有6個位元的 RAM`

`module RAM_Word6_Bit6 (CS, RW, Address, D_In, D_Out);`

`input CS, RW;`

`input [2:0] Address;`

`input [5:0] D_In;`

`output [5:0] D_Out;`

`wire [5:0] D_Out;`

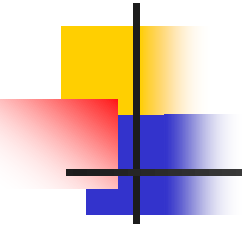
`// 引用2個具有6個字組, 每個字組有3個位元的 RAM`

`RAM_Word6_Bit3 My1_RAM_Word6_Bit3 (CS, RW, Address, D_In[2:0],
D_Out[2:0]);`

`RAM_Word6_Bit3 My2_RAM_Word6_Bit3 (CS, RW, Address, D_In[5:3],
D_Out[5:3]);`

`endmodule`

in order



Bits Extension of RAM (4)

//RAM_6_3.V：具有6個字組，每個字組有3位元的記憶體模組

```
module    RAM_Word6_Bit3 (CS, RW, Address, Data_In, Data_Out);
```

```
parameter Words = 6;
```

```
input     CS, RW;
```

```
input     [2:0] Address;
```

```
input     [2:0] Data_In;
```

```
output    [2:0] Data_Out;
```

```
reg      [2:0] RAM_Data[Words-1:0];
```

```
reg      [2:0] Data_Out;
```

```
always @(negedge CS)
```

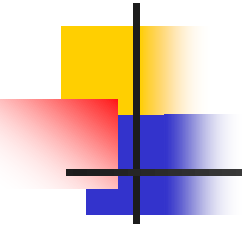
```
begin // 負緣觸發, 選擇到此晶片
```

```
    if (RW == 1'b0) // 0 = 讀取
```

```
    begin
```

```
        Data_Out = RAM_Data[Address[2:0]];
```

```
    end
```

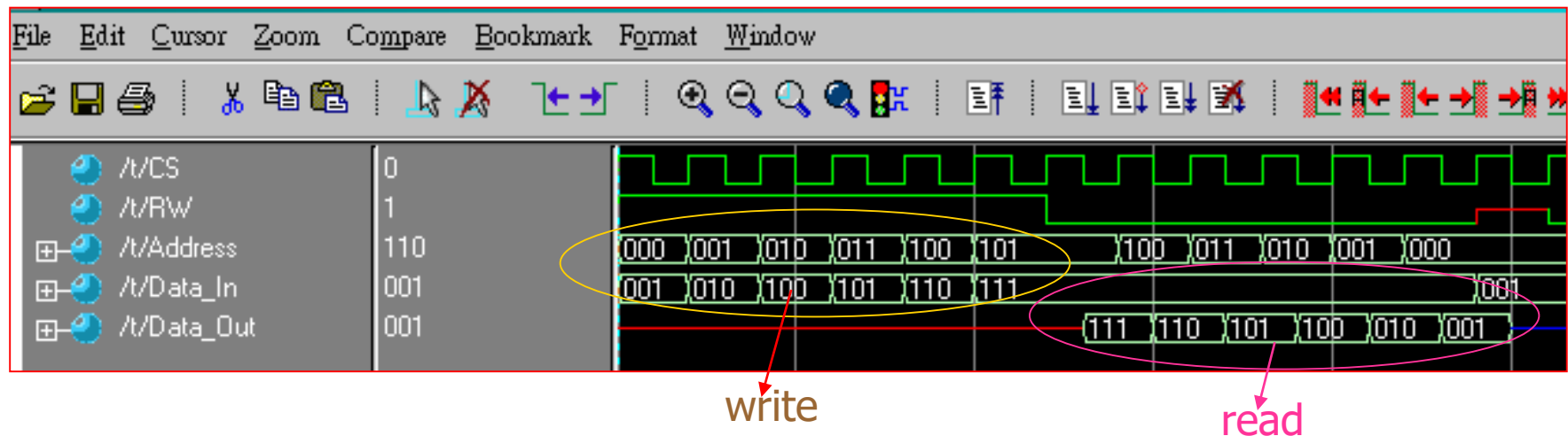



Bits Extension of RAM (5)

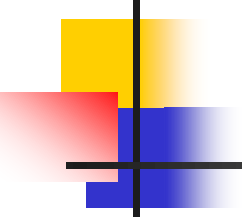
```
else
begin
    if (RW == 1'b1) // 1 = 寫入
    begin
        RAM_Data[Address[2:0]] = Data_In;
    end
    else
        Data_Out = 3'bz; // 高阻抗 (High Impedance)
    end
end
endmodule
```

Simulation of RAM_6_6.V

5_04_RAM_6_6



RAM_6_6_test.V 未附



Applications of RAM

- 隨機存取記憶體(RAM)為電路中，需要大量儲存資料的地方，而且我們可以將資料寫入到隨機存取記憶體中或是從隨機存取記憶體中讀出資料。
- 隨機存取記憶體可以用來模擬下列動作：
 - 堆疊 (Stack) 的運作
 - 佇列 (Queue) 的運作
 - 串列 (Linked List) 的運作

Function of Stack (1)

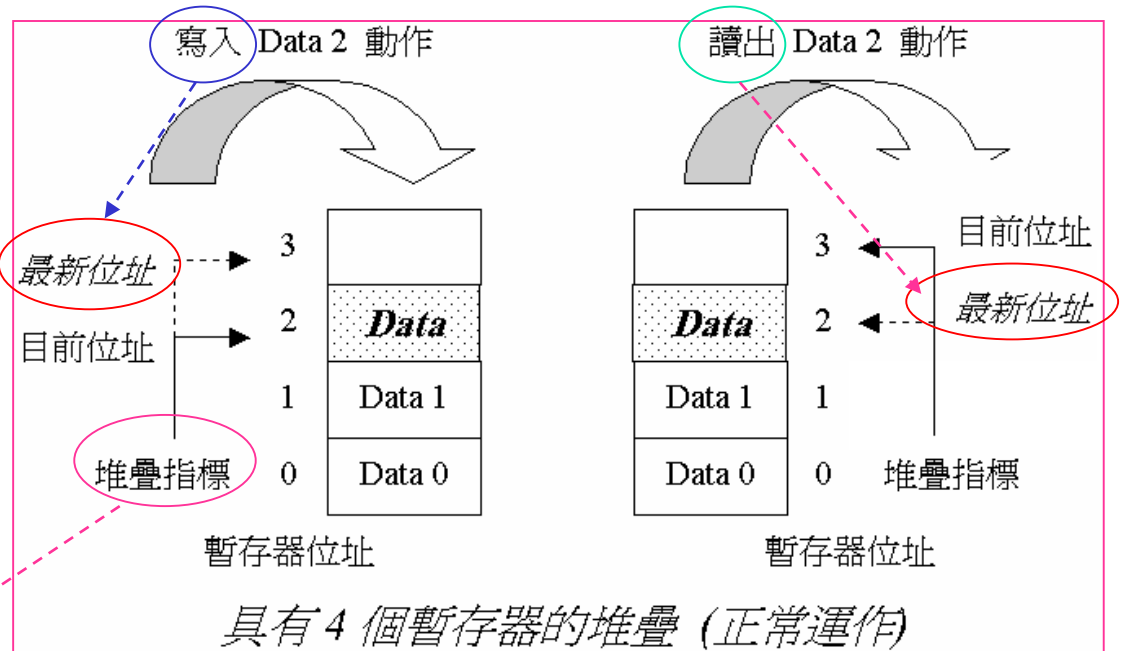
❑ 堆疊的運作：

- 堆疊 (Stack) 為一種資料結構的表示方法，它所代表的特性為：先進後出 (First In Last Out, FILO)；也就是先寫進到堆疊裡的資料最後才會被讀出來。
- 堆疊電路常被用來記錄副程式 (Sub-Routine) 或函數 (Function) 的返回位址，以及可以復原至上一個狀態的各類用途上。

❑ 我們可以使用隨機存取記憶體來模擬堆疊的運作，因為這些暫存器的位元寬度 (Bit Width) 決定了此堆疊的寬度，而這些暫存器的個數決定了此堆疊的深度 (Depth)。

❑ 堆疊內部需要有一個用於記錄“將資料寫入到堆疊中”或“從堆疊中讀出資料”的暫存器，我們稱之為「堆疊指標暫存器」(Stack Pointer, SP)。

❑ 堆疊的運作主要分為“置入資料到堆疊中 (Push)”及“從堆疊內取出資料 (Pop)”



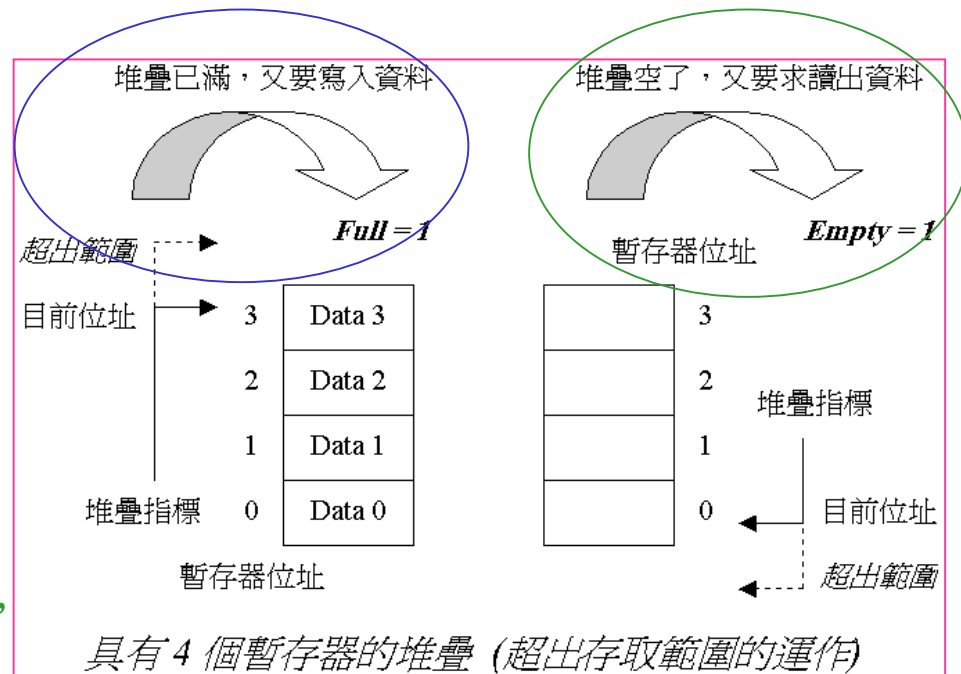
Function of Stack (2)

❑ 置入資料到堆疊中 (Push) 動作 (先寫入再加一)

- 檢查目前欲寫入資料的位址 (堆疊指標暫存器) 是不是在最大的有效位址範圍之內？
- 如果在範圍之內，則將資料寫入到堆疊指標暫存器所指的位址，而且讓堆疊指標暫存器加一 (下一個欲寫入資料的位址)。
- 如果超過了最大的有效位址範圍的話，則發出 Full = 1 (堆疊溢滿, Stack Full) 這個旗標訊號。

❑ 從堆疊內取出資料 (Pop) 動作

- 檢查目前欲讀出資料的位址 (堆疊指標暫存器) 是不是在最小的有效位址範圍之內？
- 如果在範圍之內的話，則將堆疊指標減一，再依暫存器所指的位址從堆疊內取出資料 (下一個讀出資料的位址)。
- 如果不在最小的有效位址範圍的話，則發出 Empty = 1 (堆疊空乏, Stack Empty) 這個旗標訊號。

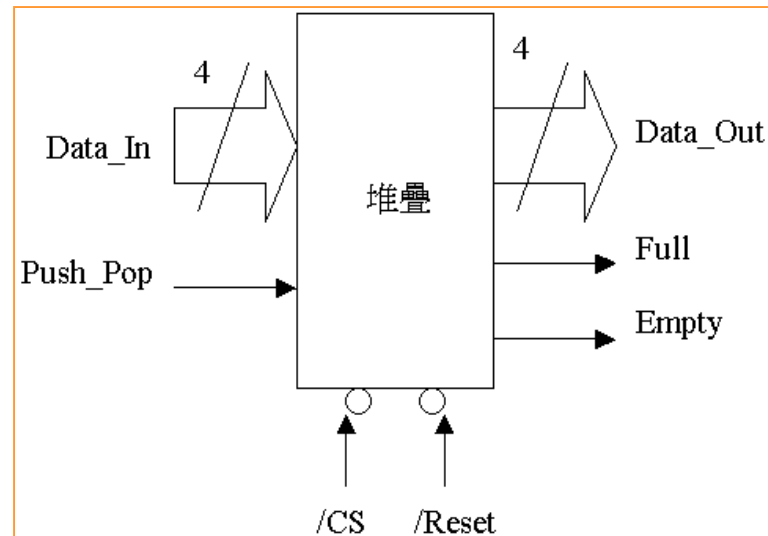


Function of Stack (3)

//StackRAM.V：堆疊(Stack)的運作，使用隨機存取記憶體來模擬

```
module Stack_By_RAM (Reset, CS, Push_Pop, Data_In,  
                    Data_Out, Full, Empty);
```

```
parameter Max_Words = 4;  
input      Reset, CS, Push_Pop;  
input [3:0] Data_In;  
output [3:0] Data_Out;  
reg [3:0] Data_Out;  
output Full, Empty;  
reg Full, Empty;
```



```
reg [3:0] RAM_Data[Max_Words-1:0];
```

```
reg [2:0] Stack_Pointer;
```

有4個word且每個word為4bit

為了判斷Max_Words, Full, and empty等5種狀況

Function of Stack (4)

always @(negedge CS)

begin // 負緣觸發:選擇到此晶片

// 預設每一個 output 的初始值, 這樣就不用在底下的每一個 if 中加入 output 的初始值。

// 預設為沒有發生: 堆疊溢滿 (Stack Full) 及 堆疊空乏 (Stack Empty) 的情形

{Full, Empty} = 2'b00;

if (Reset == 1'b0) // 電路重置

begin

Stack_Pointer[2:0] = 0; // 設定堆疊指標的位址

end

else

begin // 將 Data 置入 Stack 中 (Push 動作)

if (Push_Pop == 1'b0)

begin // 在最大有效的位址範圍內

if (Stack_Pointer[2:0] < Max_Words)

begin // 寫入 Data 到 RAM 中

RAM_Data[Stack_Pointer[2:0]] = Data_In;

Stack_Pointer[2:0] = Stack_Pointer[2:0] + 1; // 計算下一個寫入位址, 堆疊指標 +1

end

else

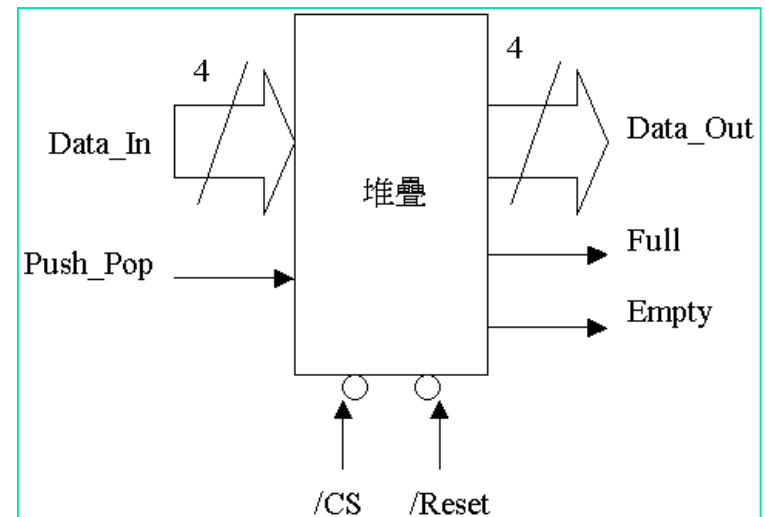
begin

Full = 1'b1; // 堆疊溢滿 (Stack Full); 即 0~3 表示未滿, 而 4 表示已滿。

end

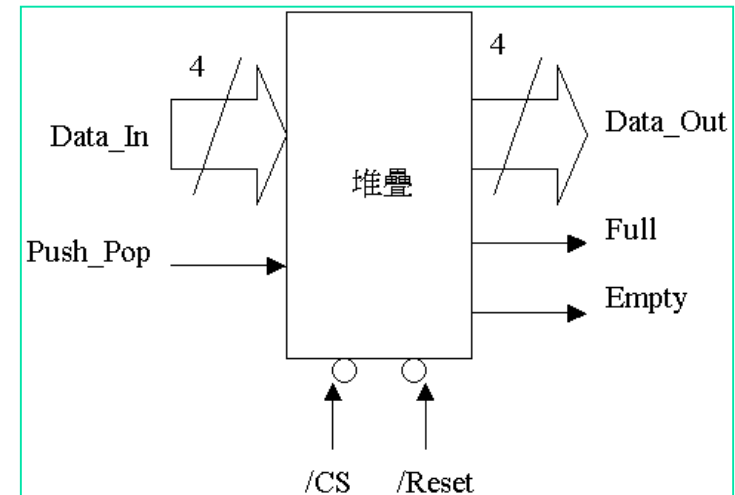
end

else



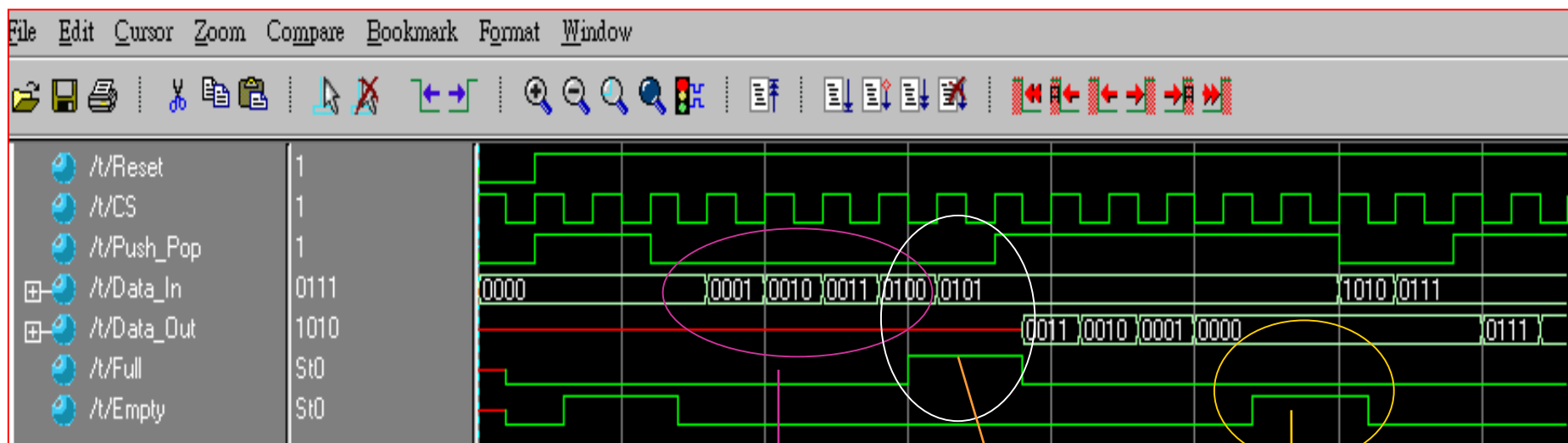
Function of Stack (5)

```
begin // 由 Stack 中取出 Data (Pop 動作)
  if (Push_Pop == 1'b1)
    begin // 在最小有效的位址範圍內
      if (Stack_Pointer[2:0] > 0) // SP在0時表示空的。
        begin // 計算取出資料的位址, 堆疊指標 -1
          Stack_Pointer = Stack_Pointer - 1;
          // 從 RAM 中讀出 Data
          Data_Out = RAM_Data[Stack_Pointer[2:0]];
        end
      else
        begin
          Empty = 1'b1; // 堆疊空乏 (Stack Empty)
        end
      end
    end
  else
    begin // Push_Pop 不為 0 (Push 動作) 或 1 (Pop 動作)
      Data_Out = 4'bz; // 高阻抗 (High Impedance)
    end
  end
end
end
endmodule
```



Simulation of StackRAM.V

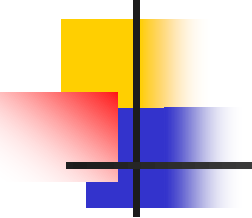
5_05_STACKRAM



做PUSH動作

Stack Fall

Stack內為空，
做pop動作
所以Empty=1



Function of Queue (1)

□ 佇列的運作：

佇列 (Queue) 為一種資料結構，它的特性為：先進先出 (First In First Out, FIFO)。

□ 佇列電路常被用來當作資料的緩衝器 (Data Buffer) 以及事件的排程(Event Schedule) 等用途上。

➤ 當作資料緩衝器使用時，可使資料之間的傳輸速度維持在一個較平穩的狀態；例如：網路介面卡上的緩衝器、印表機的緩衝器、CPU 內部的指令緩衝器 ... 等等。

➤ 當作事件的排程使用時，可以將事件發生的順序依序地記錄下來，再依照欲處理的順序依序地處理。

□ 佇列內部有二個位址暫存器：

➤ 一個用於記錄從佇列中讀出資料的指標 (資料讀出指標, Tail Pointer)

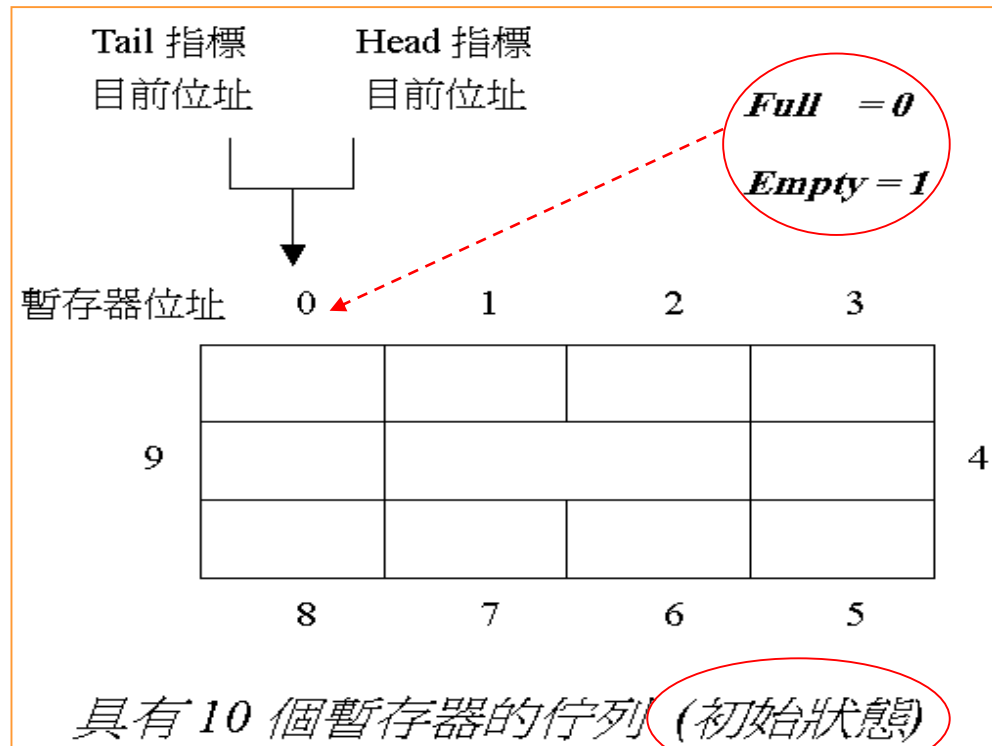
➤ 另一個則是用於記錄將資料寫入到佇列中的指標 (資料寫入指標, Head Pointer)。

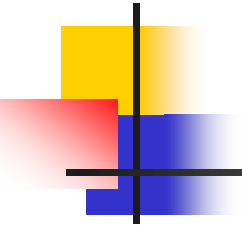
➤ 資料讀出指標所指的位址為本次的讀出操作時，該被讀出資料的位址。

➤ 資料寫入指標所指的位址為下次的寫入操作時，該被寫入資料的位址。

Function of Queue (2)

- 佇列的運作主要有：
- 置入資料到佇列中 (Insert)
 - 從佇列內取出資料 (Delete)。
 - 配合佇列的狀態旗標來檢知，佇列是否溢滿不能再寫入資料的 Full 旗標，以及佇列是否空乏沒有資料可供讀出的 Empty 旗標。
 - 佇列的詳細運作方式請看下圖的數個「具有10個暫存器的佇列」圖所示。





Function of Queue (3)

□ 佇列的運作方式：

➤ 佇列的初始狀態

- ✓ 一開始佇列的資料讀出指標 (Tail) 與資料寫入指標 (Head) 的目前位址均指向暫存器位址0，佇列溢滿 Full 旗標設為0 (代表佇列仍可寫入資料)，以及佇列空乏 Empty 旗標設為1 (代表佇列內沒有資料可供讀出)。

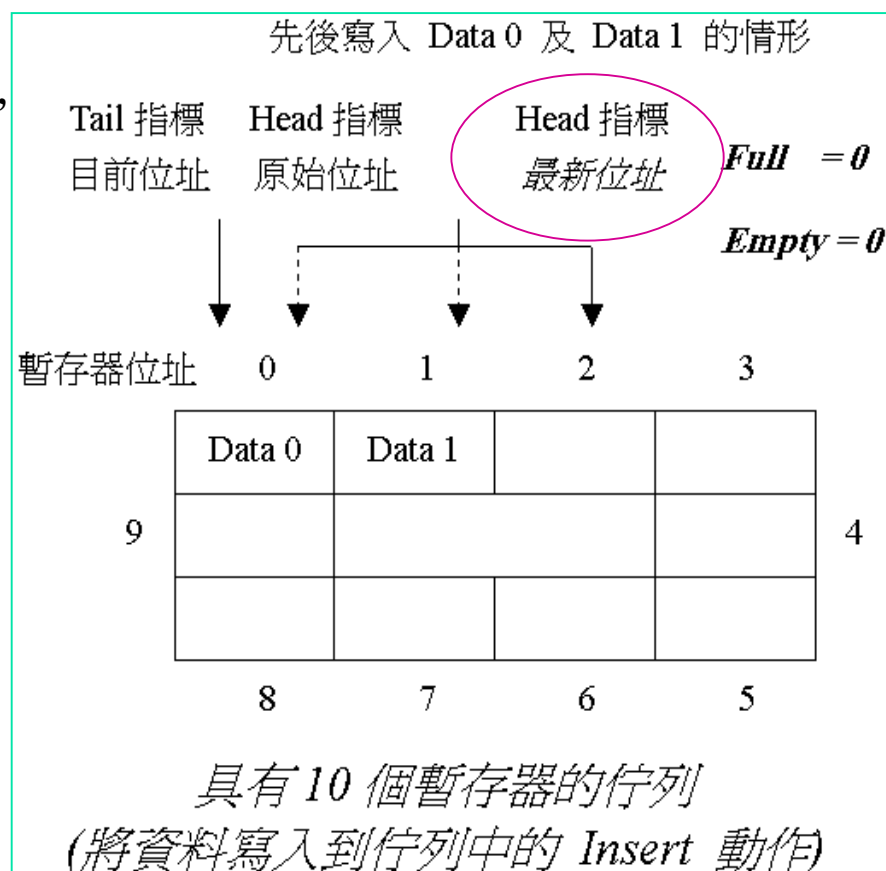
➤ 將資料寫入到佇列中的 Insert 動作

- ✓ 如果佇列是空的 (Empty 旗標為1、沒有資料可供讀出)，佇列的 Insert 動作才能動作。
- ✓ 將資料寫入到資料寫入指標 (Head Pointer) 所指的位址。
- ✓ 同時讓資料寫入指標 (Head Pointer) 加一 (下一個欲寫入資料的位址)
- ✓ 如果寫入指標 (Head Pointer) 超過了最大的有效位址範圍的話，則將資料寫入指標 (Head Pointer) 設為最小的有效位址。
- ✓ 最後將佇列空乏 Empty 旗標設為0 (代表佇列內有資料可供讀出)。

Function of Queue (4)

- ✓ 如果佇列目前不是空乏的 (Empty 旗標為0、有資料可供讀出)，Insert 動作，會先比較資料寫入指標 (Head Pointer) 與資料讀出指標 (Tail) 是否相同？如果相同，則將佇列溢滿 Full 旗標設為1 (代表佇列已滿、無法再將資料寫入到佇列中)。如果不同，則將資料寫入到資料寫入指標 (Head Pointer) 所指的位址。
- ✓ 讓資料寫入指標 (Head Pointer) 加一 (下一個欲寫入資料的位址)，如果寫入指標 (Head Pointer) 超過了最大的有效位址範圍的話，則將資料寫入指標 (Head Pointer) 設為最小的有效位址。

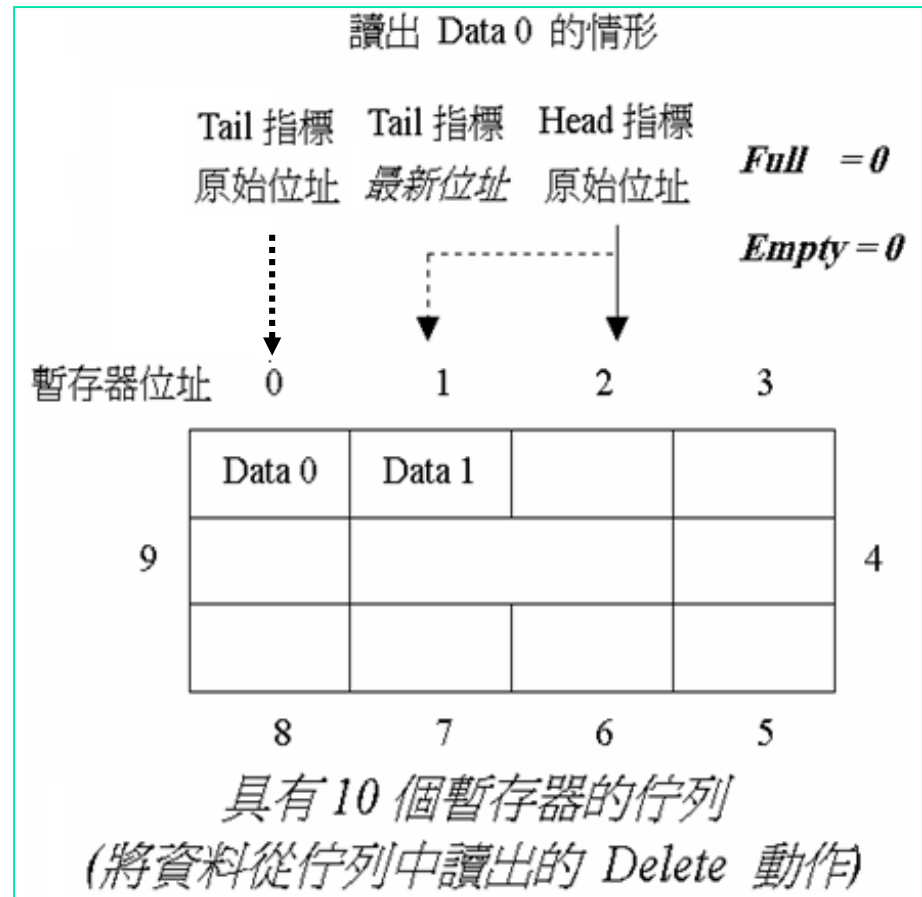
- 一開始 Empty = 1，將 Data0 寫入 Head Pointer 所指位址並將 Head Pointer + 1 且 Empty 設為 0；
- 之後先判斷 Head Pointer 與 Tail pointer 是否相同，如果是則 Full 設為 1，
- 如果不是，將 Data0 寫入 Head Pointer 所指位址並將 Head Pointer + 1
- 且如果 Head Pointer 超過最大有效位址則將 Head Pointer 設為最小有效位址，以此類推。



Function of Queue (5)

❑ 將資料從佇列中讀出的 Delete 動作

- 如果佇列目前是空乏的 (Empty 旗標為1、沒有資料可供讀出)，在此狀態下佇列的 Delete 動作，將無法從佇列中讀出資料。
- 如果佇列目前不是空乏的 (Empty 旗標為0、有資料可供讀出)，Delete 動作，會依序作如下的處理：
 - ✓ 依資料讀出指標 (Tail Pointer) 所指向佇列中的位址讀出資料。
 - ✓ 同時讓資料讀出指標 (Tail Pointer) 加一 (下一個欲讀出資料的位址)。
 - ✓ 如果資料讀出指標 (Tail Pointer) 超過了最大的有效位址範圍的話，則將資料讀出指標 (Tail Pointer) 設為最小的有效位址。

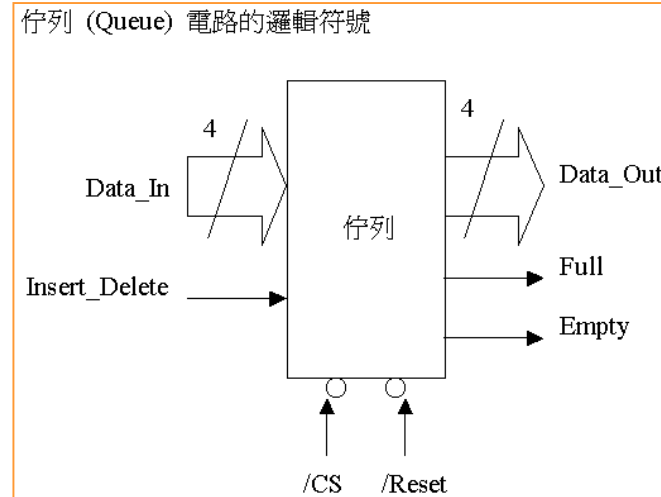


Function of Queue (6)

//QueueRAM.V：佇列的運作，使用隨機存取記憶體來模擬

```
module Queue_By_RAM (Reset, CS, Insert_Delete, Data_In, Data_Out, Full,  
    Empty);
```

```
parameter    Max_Words = 4;  
input        Reset, CS, Insert_Delete;  
input        [3:0] Data_In;  
output       [3:0] Data_Out;  
reg          [3:0] Data_Out;  
output       Full, Empty;  
reg          Full, Empty;
```



```
reg [3:0] RAM_Data[Max_Words-1:0];
```

```
reg [1:0] Queue_Head_Pointer, Queue_Tail_Pointer; //最大為4
```

4個word每個word為4bit

$\begin{cases} \text{Write - in} \Rightarrow \text{From Head - pointer} \\ \text{Read - out} \Rightarrow \text{From Tail - pointer} \end{cases}$

Function of Queue (7)

always @(negedge CS)

begin // 負緣觸發: 重置 或 選擇到此晶片

if (Reset == 1'b0) // 電路重置

begin

Queue_Head_Ponter[1:0] = 0; // 設定: 佇列指標開頭的位址

Queue_Tail_Ponter[1:0] = 0; // 設定: 佇列指標結尾的位址

Full = 1'b0; // 佇列'未'溢滿

Empty = 1'b1; // 佇列空乏

end

else

begin // 將 Data 置入 Queue 中 (Insert 動作)

if (Insert_Delete == 1'b0)

begin // 佇列 '未' 溢滿 且 佇列是空的

if ((Full == 1'b0) && (Empty == 1'b1))

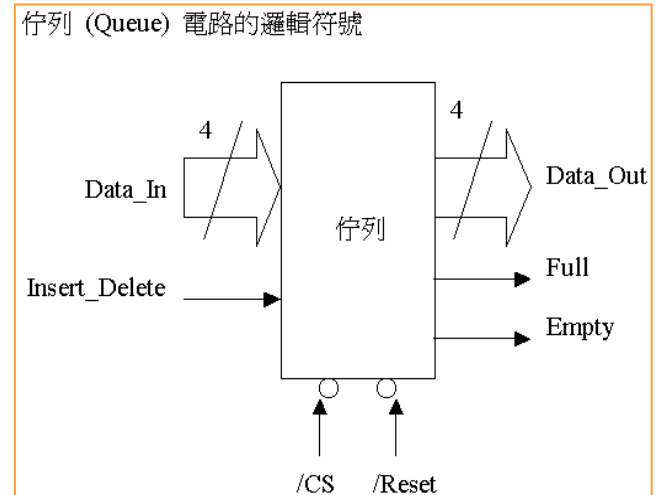
begin // 寫入 Data 到 RAM 中

RAM_Data[Queue_Head_Ponter[1:0]] = Data_In;

Queue_Head_Ponter[1:0] = Queue_Head_Ponter[1:0] + 1;

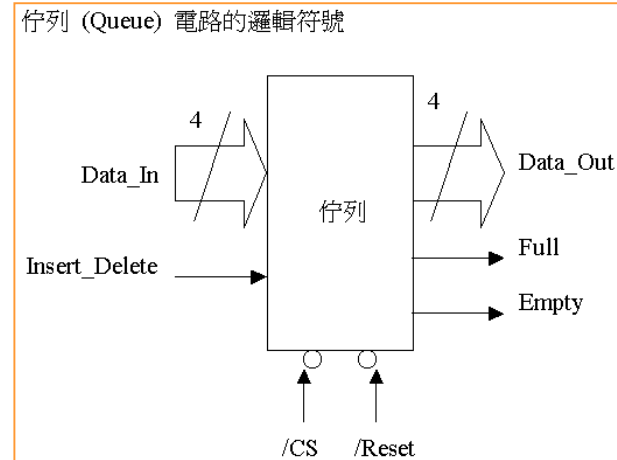
if (Queue_Head_Ponter[1:0] == Max_Words)

begin



Function of Queue (8)

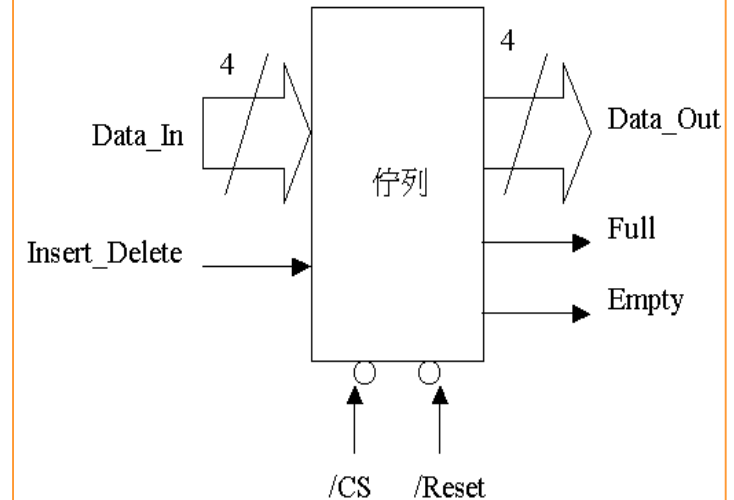
```
Queue_Head_Pointer[1:0] = 0;
end
Empty = 1'b0; // 佇列'未'空乏
end
else
begin // 佇列'未'溢滿 以及 佇列'未'空乏
    if ((Full == 1'b0) && (Empty == 1'b0))
    begin
        if (Queue_Head_Pointer[1:0] != Queue_Tail_Pointer[1:0])
        begin // 寫入 Data 到 RAM 中
            RAM_Data[Queue_Head_Pointer[1:0]] = Data_In;
            // 計算下一個寫入資料的位址, 寫入資料指標 +1
            Queue_Head_Pointer[1:0] = Queue_Head_Pointer[1:0] + 1;
            if (Queue_Head_Pointer[1:0] == Max_Words)
            begin
                Queue_Head_Pointer[1:0] = 0; // 清空並註明佇列已滿
            end
        end
    end
end
else
```



Function of Queue (9)

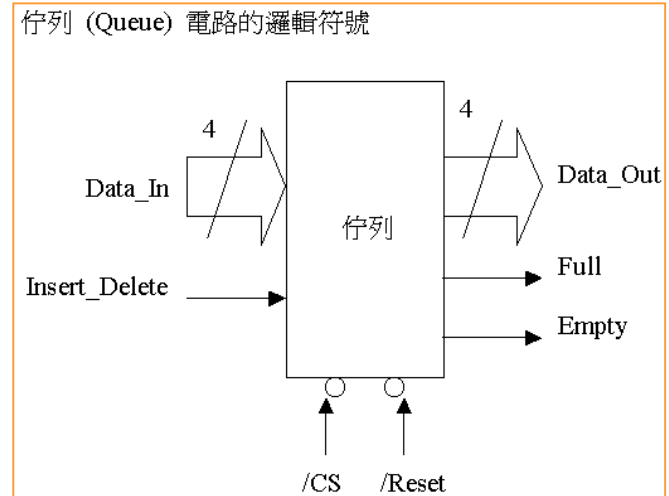
```
begin
    Queue_Head_Pointer[1:0] = Queue_Tail_Pointer[1:0]
    Full = 1'b1; // 佇列溢滿 (Queue Full)
end
end
end
else
begin // 從 Queue 中取出 Data (Delete 動作)
    if (Insert_Delete == 1'b1)
    begin // 佇列'未'溢滿 以及 佇列空乏
        if (Empty == 1'b0) // 佇列'未'空乏
        begin // 從 RAM 中讀出 Data
            Data_Out = RAM_Data[Queue_Tail_Pointer[1:0]];
            // 計算下一個資料的讀出位址, 讀出資料指標 +1
            Queue_Tail_Pointer[1:0] = Queue_Tail_Pointer[1:0] + 1;
            if (Queue_Tail_Pointer[1:0] == Max_Words) // 表示超過0~3
            begin
                Queue_Tail_Pointer[1:0] = 0; // 指向TP的最小值
            end
        end
    end
end
```

佇列 (Queue) 電路的邏輯符號



Function of Queue (10)

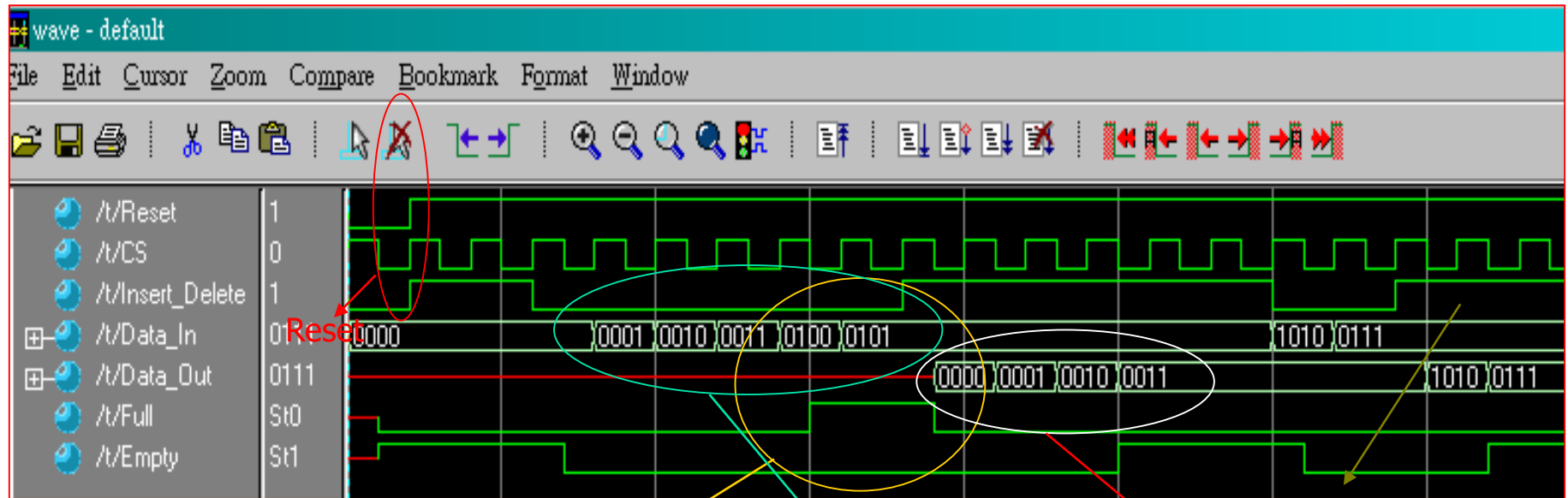
```
Full = 1'b0; // 佇列'未'溢滿但為空的
if (Queue_Tail_Pointer[1:0] != Queue_Head_Pointer[1:0])
begin
    Empty = 1'b0; // 佇列'未'空乏
end
else
begin
    Empty = 1'b1; // 佇列空乏 (Queue Empty)
end
end
end
else
begin // Insert_Delete 不為 0 (Insert 動作) 或 1 (Delete 動作)
    Data_Out = 4'bz; // 高阻抗 (High Impedance)
end
end
end
end
endmodule
```



Queue_Tail_Pointer[1:0] = Queue_Head_Pointer[1:0]

Simulation of QueueRAM.V

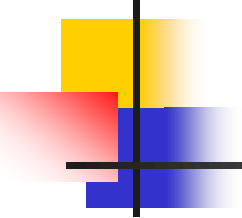
5_06_QUEUERAM



Empty = 1
做Delete(讀出)動作
所以Data_Out為
High Impedance

做Insert(寫入)動作
將0, 1, 2, 3依序寫入
並依序將Head_Pointer+1

做Delete(讀出)動作
0, 1, 2, 3依序讀出
並依序將Tail_Pointer+1

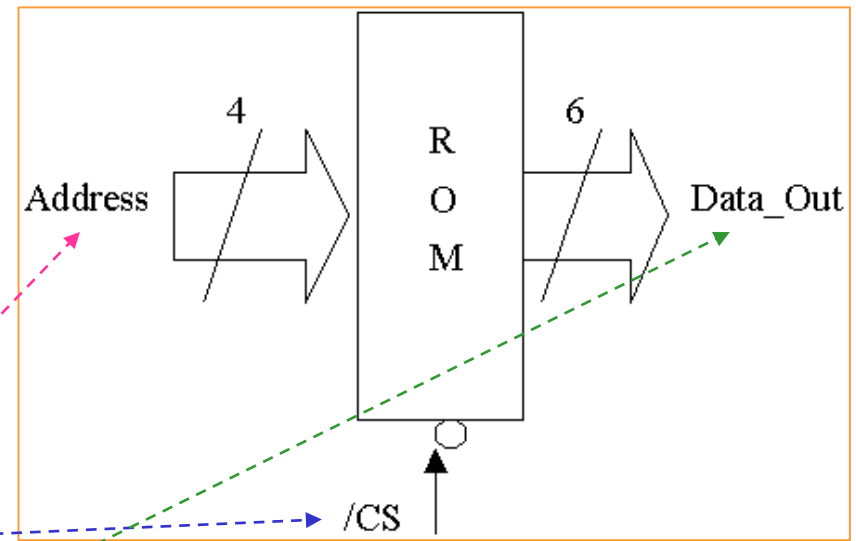


Verilog 硬體描述語言 (I)

- ☐ 隨機存取記憶體(RAM)
- ☐ 唯讀記憶體(ROM)

ROM (1)

- ❑ 唯讀記憶體(ROM)用於儲存固定不會再更動的值，它由一些基本的電晶體 (Transistor)、位址解碼器 (Address Decoder) 與讀取動作的邏輯閘電路所組成的。
- ❑ 電晶體被連接到 Vdd 與 Vss，其中 Vdd 代表固定為1，而 Vss 固定為0。
- ❑ 通常唯讀記憶體(ROM)都有下列的控制訊號: CS (晶片選擇, Chip Select)、Address (位址線) 以及 Data_Out (資料輸出線)。



ROM (2)

// ROMW16B6.V：具有16個字組, 每個字組有6個位元的唯讀記憶體

```
module ROM (CS, Address, D_Out);
```

```
input      CS;
```

```
input      [3:0] Address;
```

```
output     [5:0] D_Out;
```

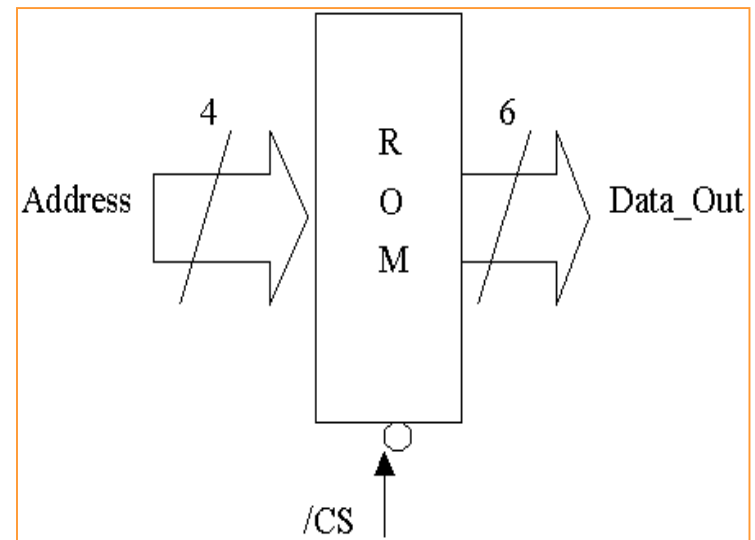
```
reg        [5:0] D_Out;
```

// 使用 16個 6 Bits 的 Register來模擬 ROM

```
reg        [5:0] ROM_Data[15:0];
```

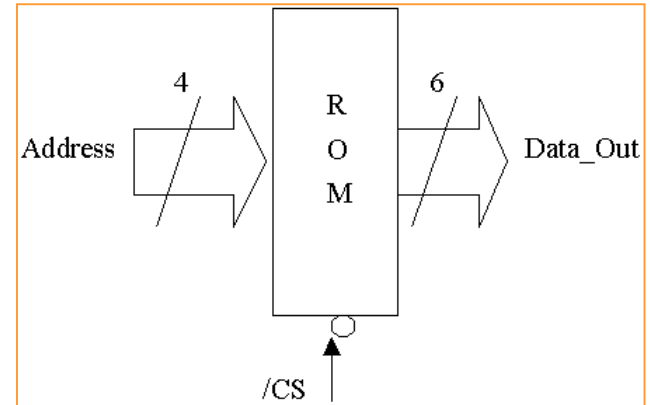
```
always @(negedge CS) // 選擇到此晶片
```

```
begin
```



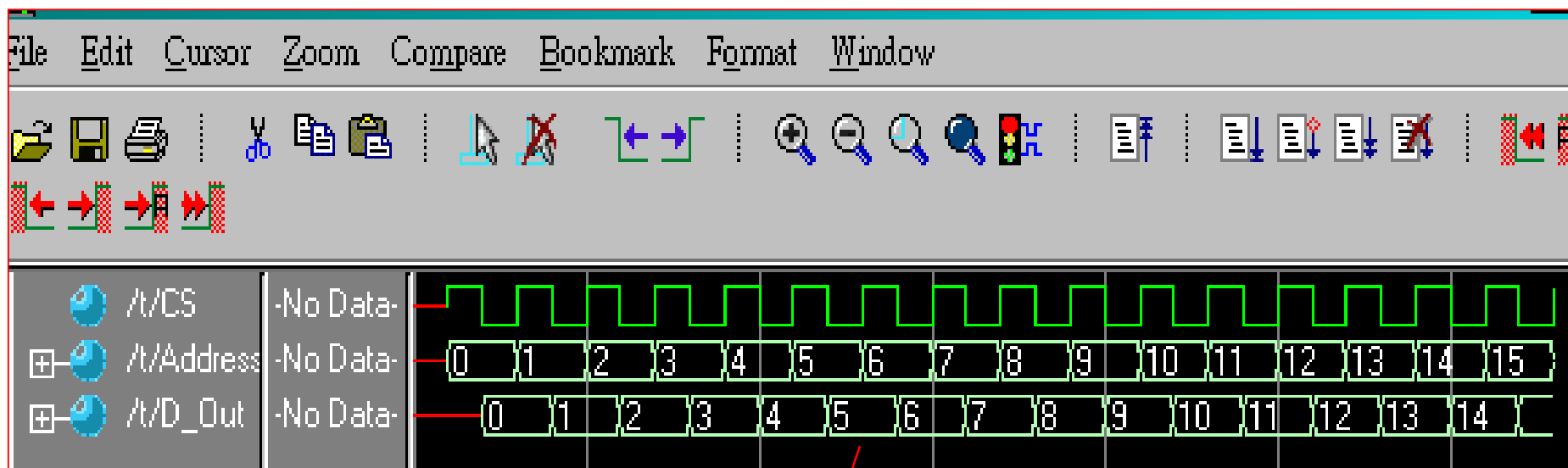
ROM (3)

```
ROM_Data [ 0] = 6'd0; // 讓位址為0的內容固定為0
ROM_Data [ 1] = 6'd1; // 讓位址為1的內容固定為1
ROM_Data [ 2] = 6'd2; // 讓位址為2的內容固定為2
ROM_Data [ 3] = 6'd3; // 讓位址為3的內容固定為3
ROM_Data [ 4] = 6'd4; // 讓位址為4的內容固定為4
ROM_Data [ 5] = 6'd5; // 讓位址為5的內容固定為5
ROM_Data [ 6] = 6'd6; // 讓位址為6的內容固定為6
ROM_Data [ 7] = 6'd7; // 讓位址為7的內容固定為7
ROM_Data [ 8] = 6'd8; // 讓位址為8的內容固定為8
ROM_Data [ 9] = 6'd9; // 讓位址為9的內容固定為9
ROM_Data[10] = 6'd10; // 讓位址為10的內容固定為10
ROM_Data[11] = 6'd11; // 讓位址為11的內容固定為11
ROM_Data[12] = 6'd12; // 讓位址為12的內容固定為12
ROM_Data[13] = 6'd13; // 讓位址為13的內容固定為13
ROM_Data[14] = 6'd14; // 讓位址為14的內容固定為14
ROM_Data[15] = 6'd15; // 讓位址為15的內容固定為15
D_Out = ROM_Data[Address]; // 由 ROM_Data 內取出資料
end
endmodule
```



Simulation of ROMW16B6.V

5_07_ROMW16B6

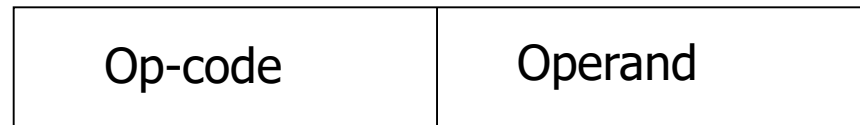


將位址與Data讀出

Application of ROM

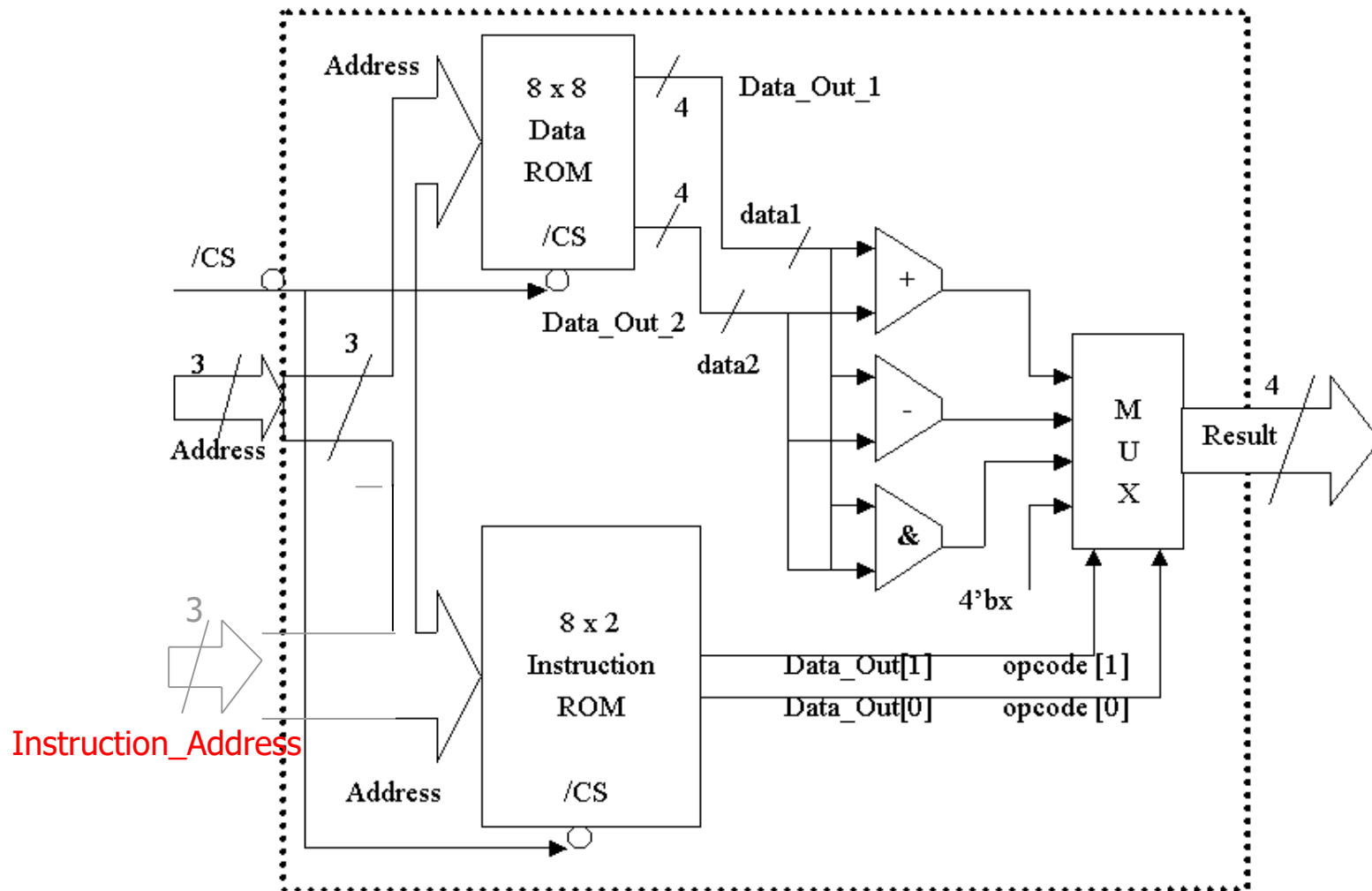
- ❑ 唯讀記憶體 (ROM) 為電路中需要大量儲存資料的地方，而且這些資料只供讀取而無法寫入。
- ❑ 唯讀記憶體在電路中廣泛地被使用到，例如：早期主機板上的 BIOS (Basic Input / Output System)、網路卡上的 BIOS、印表機內建的字型 ROM ... 等應用。
- ❑ 範例：將 ROM 內存放在 '指令' 的「指令唯讀記憶體」(Instruction ROM) 及將 ROM 內存放在 '資料' 的「資料唯讀記憶體」(Data ROM)。
 - 指令唯讀記憶體 (Instruction ROM) 和一般的唯讀記憶體 (ROM) 的基本電路架構是一樣的，即某一顆 ROM 內存放的資料可用來作為某種特定算術單元的「指令碼」則稱此顆 ROM 為指令唯讀記憶體 (Instruction ROM)。
 - 資料唯讀記憶體 (Data ROM) 也和一般的唯讀記憶體 (ROM) 的電路架構是一樣的，即某一顆 ROM 內存放的資料可用來作為某種特定算術單元的「資料碼」則稱此顆 ROM 為資料唯讀記憶體 (Data ROM)。

Instruction Format :



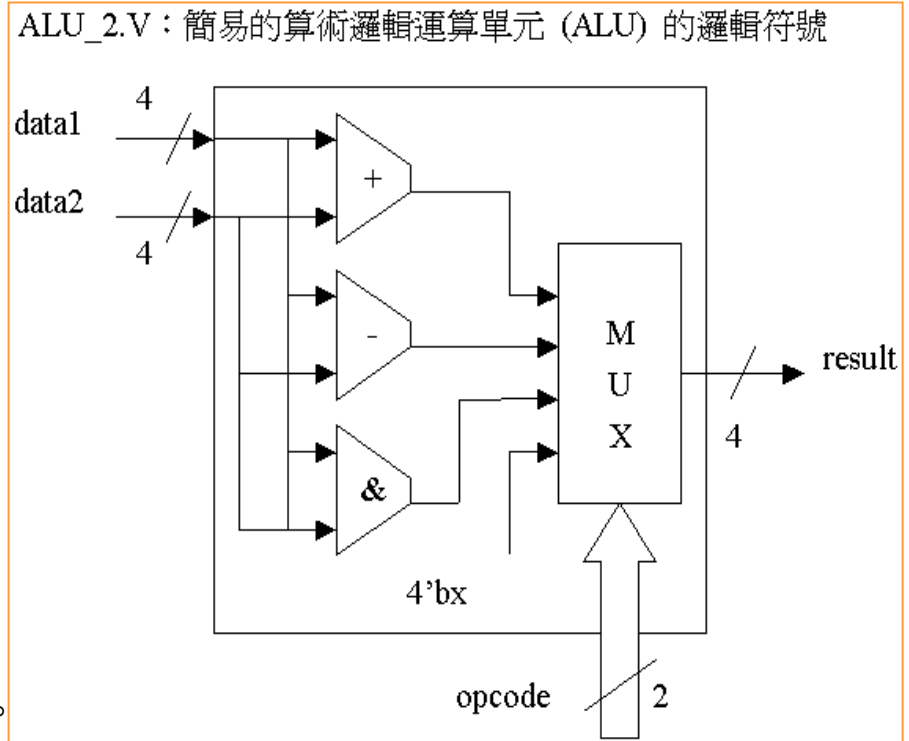
Appendix: ROM_ALU Scheme

ROM_ALU.V的邏輯方塊圖



ROM_ALU.V (1)

- ❑ 算術邏輯運算單元 (ALU) 結合了數位系統中最基本的：加法、減法、AND ... 等等的運算，由運算碼 (operator) 來決定您想對輸入的資料作那一種的運算。
- ❑ 範例：因為我們要設計的模組需要引用一顆指令 ROM (Inst_ROM.V) 與一顆資料 ROM (Data_ROM.V) 以及一個簡易的算術邏輯運算單元 (ALU_2.V)，因此我們利用 ``include` 編譯命令來引用設計好的這些模組。
- ❑ 程式 (ALU_2.V)：算術邏輯運算單元在 ALU_2.V 中，運算碼 (operator) 可以是加法、減法、AND ... 等其中一種，只要是輸入的資料或者是運算碼 (operator) 有任何的改變後，就會重新計算。



編譯命令，定義Plus、Minus、Bin_And變數值

ALU_2.V：簡易的算術邏輯運算單元 (ALU) 的邏輯符號

The diagram illustrates the logic symbol for the ALU_2.V component. It features two 4-bit data inputs, `data1` and `data2`, and a 2-bit `opcode` input. Inside the ALU block, the data inputs are connected to three parallel logic units: an adder (+), a subtractor (-), and an AND gate (&). The outputs of these units are connected to a 4-bit Multiplexer (MUX). The `opcode` input selects the output of the MUX. The final output is a 4-bit `result`.

觸發事件為輸入訊號

ROM_ALU.V (3)

```
case (opcode)
```

```
    // Arithmetic operations
```

```
    `Plus:
```

```
        result = data1 + data2;
```

```
    `Minus:
```

```
        result = data1 - data2;
```

```
    // Bitwise operations
```

```
    `Bin_And:
```

```
        result = data1 & data2;
```

```
    default:
```

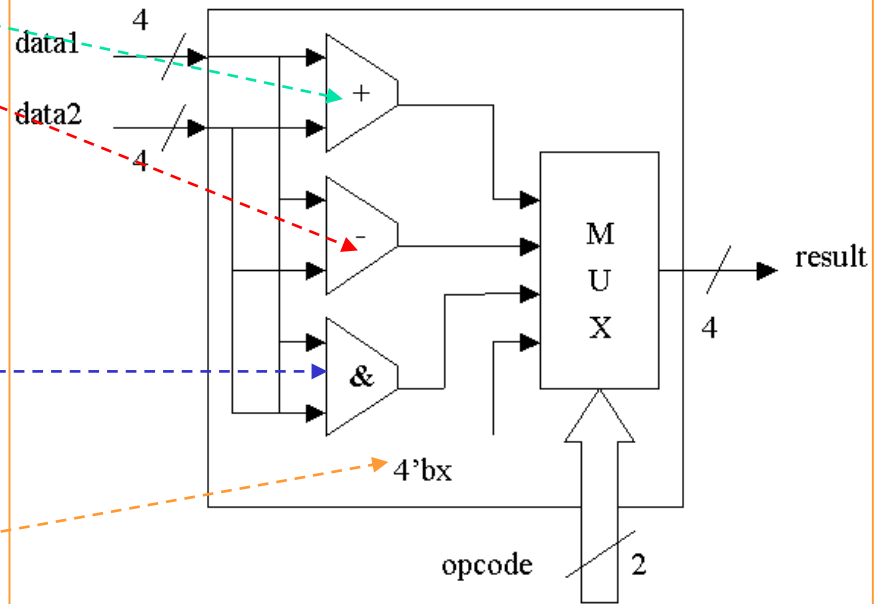
```
        result = 4'hx;
```

```
    endcase
```

```
end
```

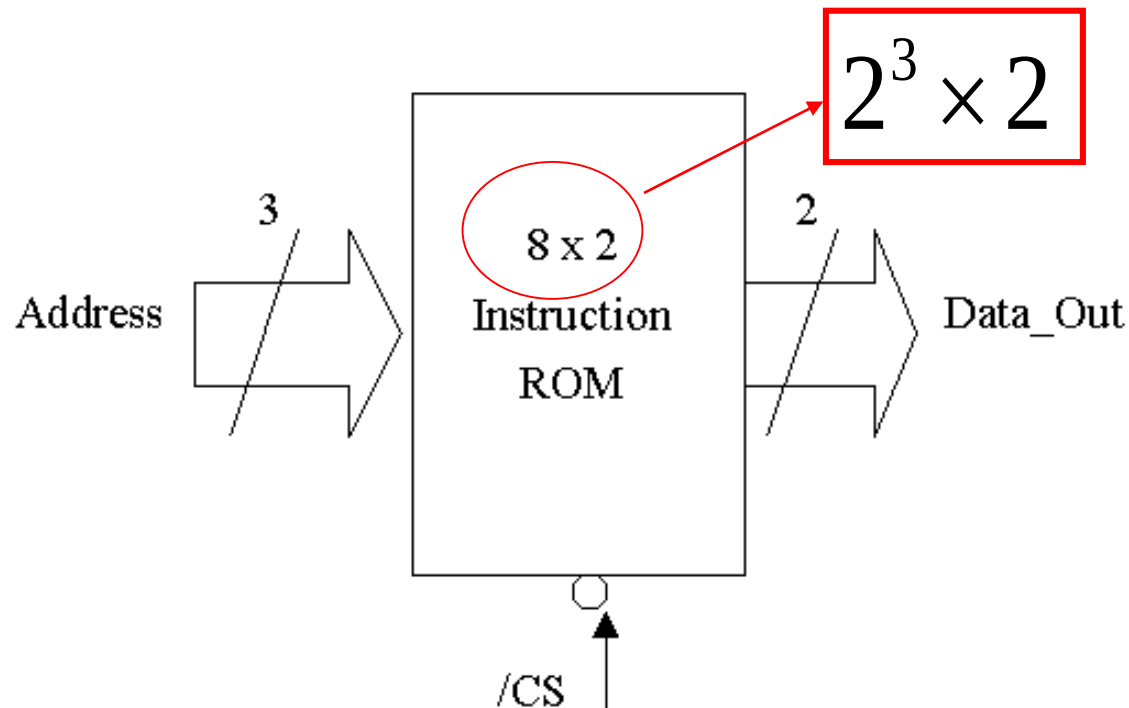
```
endmodule
```

ALU_2.V: 簡易的算術邏輯運算單元 (ALU) 的邏輯符號



ROM_ALU.V (4)

- Inst_ROM.V：存放8個字組，每個字組有2個位元的‘指令’唯讀記憶體。
- 我們將：2'b00、2'b01、2'b10、2'b11、2'b00、2'b01、2'b10以及2'b11等8個字組存放在 Inst_ROM.V 中，作為推動 ALU_2.V 簡易的算術邏輯運算單元 (ALU) 的指令碼來源。



ROM_ALU.V (5)

//Inst_ROM.V：存放8個字組，每個字組有2個位元的‘指令’唯讀記憶體。

```
module Inst_ROM (CS, Address, Data_Out);
```

```
input    CS;
```

```
input    [2:0] Address;
```

```
output    [1:0] Data_Out;
```

```
reg       [1:0] Data_Out;
```

// 使用 8個 2 Bits 的 Register, 來模擬 ROM

```
reg       [1:0] ROM_Data[7:0];
```

```
always @(negedge CS) // 選擇到此晶片
```

```
begin
```

```
    ROM_Data [0] = 2'b00; // 讓位址為0的內容固定為 2'b00, `Plus
```

```
    ROM_Data [1] = 2'b01; // 讓位址為1的內容固定為 2'b01, `Minus
```

```
    ROM_Data [2] = 2'b10; // 讓位址為2的內容固定為 2'b10, `Bin_And
```

```
    ROM_Data [3] = 2'b11; // 讓位址為3的內容固定為 2'b11, Non-Support
```

```
    ROM_Data [4] = 2'b00; // 讓位址為4的內容固定為 2'b00, `Plus
```

```
    ROM_Data [5] = 2'b01; // 讓位址為5的內容固定為 2'b01, `Minus
```

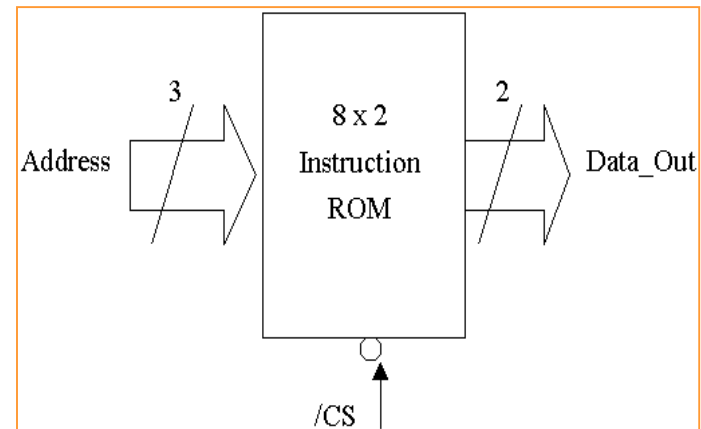
```
    ROM_Data [6] = 2'b10; // 讓位址為6的內容固定為 2'b10, `Bin_And
```

```
    ROM_Data [7] = 2'b11; // 讓位址為7的內容固定為 2'b11, Non-Support
```

```
    Data_Out = ROM_Data[Address]; // 由 ROM_Data 內取出資料
```

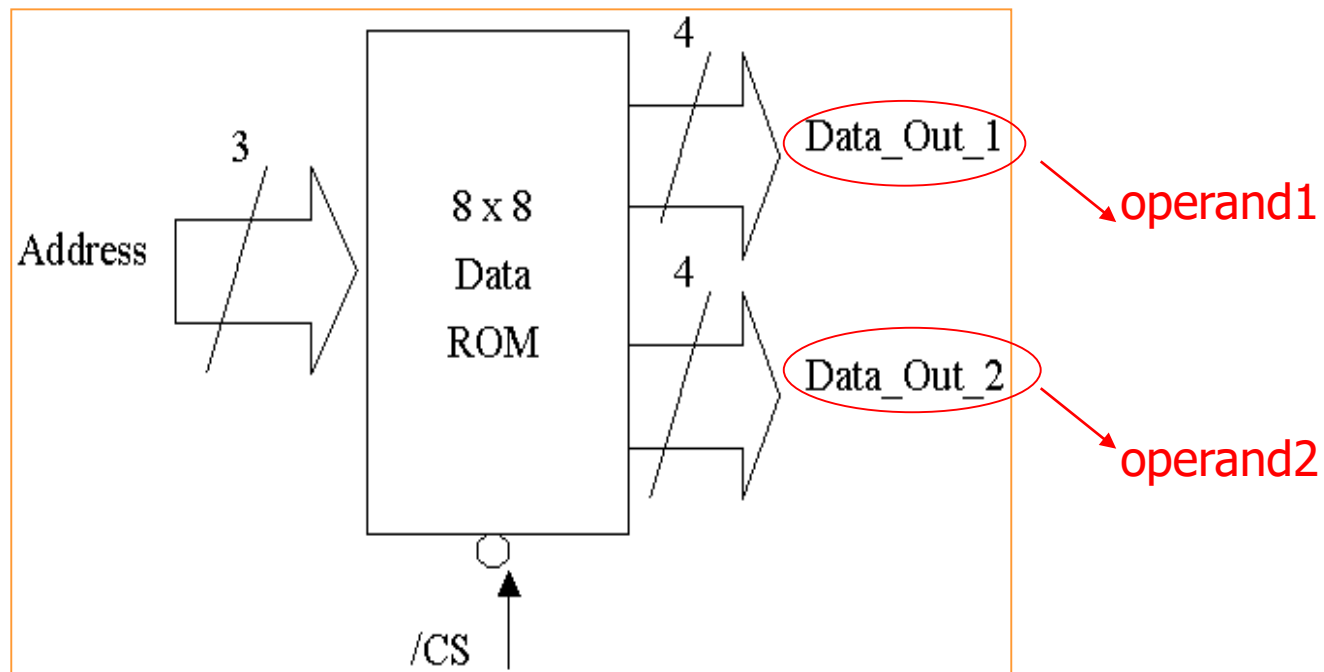
```
end
```

```
endmodule
```



ROM_ALU.V (6)

- ❑ Data_ROM.V：存放8個字組，每個字組有8個位元的‘資料’唯讀記憶體。
- ❑ 我們將：0x55、0xA5、0xAA、0x00、0xFF、0x0F、0xF0以及0x5A等8個字組存放在Data_ROM.V中，作為推動 ALU_2.V簡易的算術邏輯運算單元 (ALU) 的資料碼來源。
- ❑ 這顆 ROM 和 Inst_ROM.V 的 ROM 不同的地方在於：它有二個4個位元的資料輸出埠 (Data_Out_1 [3:0] 及 Data_Out_2 [2:0])，而 Inst_ROM.V 的 ROM 只有一個2個位元的資料輸出埠 (Data_Out [1:0])。



ROM_ALU.V (7)

//Data_ROM.V：存放8個字組, 每個字組

//有8個位元的‘資料’唯讀記憶體

```
module Data_ROM (CS, Address,  
                 Data_Out1, Data_Out2);
```

```
input    CS;
```

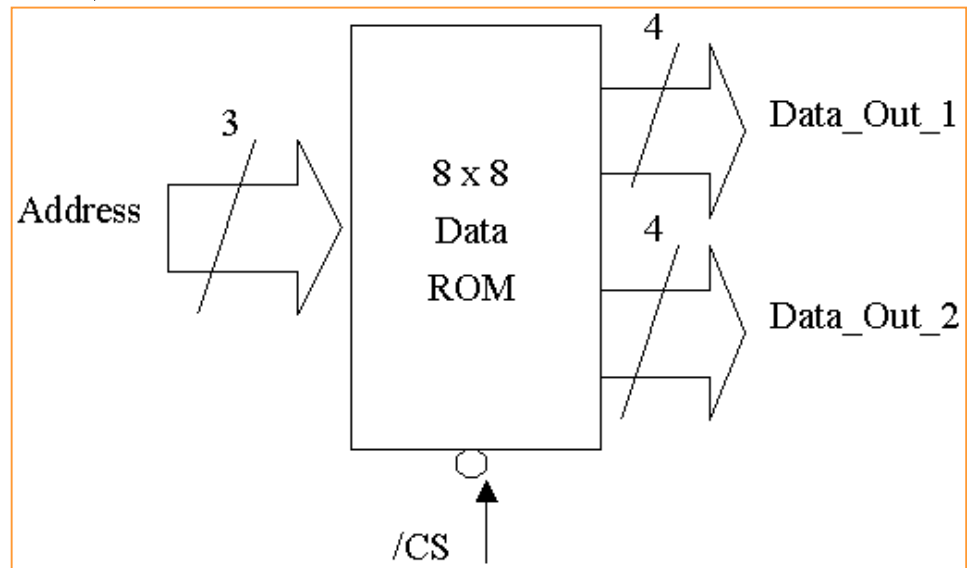
```
input    [2:0] Address;
```

```
output   [3:0] Data_Out1;
```

```
reg      [3:0] Data_Out1;
```

```
output   [3:0] Data_Out2;
```

```
reg      [3:0] Data_Out2;
```



// 使用 8 個 8 Bits 的 Register, 來模擬 ROM

```
reg      [7:0] ROM_Data[7:0];
```

```
reg      [7:0] Data_Out;
```

ROM_ALU.V (8)

always @(negedge CS) // 選擇到此晶片

begin

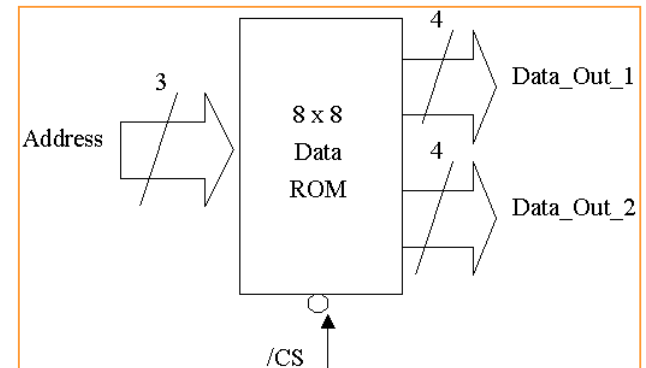
```
ROM_Data [0] = 8'b0101_0101; // 讓位址為0的內容固定為 0x55
ROM_Data [1] = 8'b1010_0101; // 讓位址為1的內容固定為 0xA5
ROM_Data [2] = 8'b1010_1010; // 讓位址為2的內容固定為 0xAA
ROM_Data [3] = 8'b0000_0000; // 讓位址為3的內容固定為 0x00
ROM_Data [4] = 8'b1111_1111; // 讓位址為4的內容固定為 0xFF
ROM_Data [5] = 8'b0000_1111; // 讓位址為5的內容固定為 0x0F
ROM_Data [6] = 8'b1111_0000; // 讓位址為6的內容固定為 0xF0
ROM_Data [7] = 8'b0101_1010; // 讓位址為7的內容固定為 0x5A
Data_Out = ROM_Data[Address]; // 由 ROM_Data 內取出資料
Data_Out1[3:0] = Data_Out[7:4]; // 取出位元 [7:4] 給 Data_Out1
Data_Out2[3:0] = Data_Out[3:0]; // 取出位元 [3:0] 給 Data_Out2
```

end

endmodule

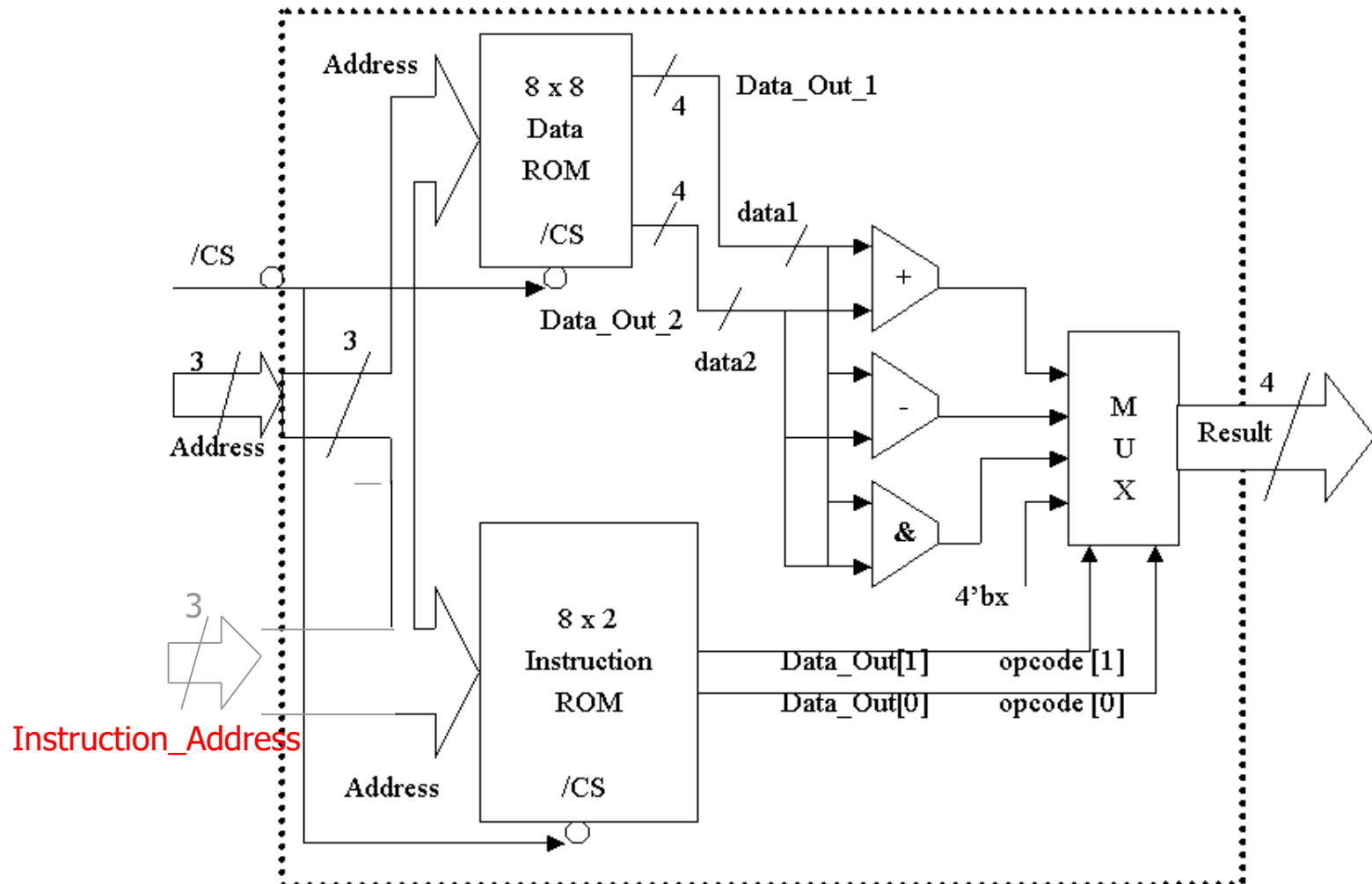
可改寫成

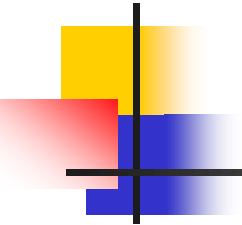
{Data_Out1, Data_Out2} = ROM_Data[Address]



ROM_ALU.V (9)

ROM_ALU.V的邏輯方塊圖





ROM_ALU.V (10)

//ROM_ALU.V：使用指令 ROM 與資料 ROM 來推動 ALU

// Inst_ROM.V, 存放8個字組, 每個字組有2個位元的'指令'唯讀記憶體

// Data_ROM.V, 存放8個字組, 每個字組有8個位元的'資料'唯讀記憶體

// ALU_2.V, 簡易的算術邏輯運算單元 (ALU)

// 引用 Inst_ROM.V, Data_ROM.V 以及 ALU_2.V

`include "Inst_ROM.V" // 8x2 的 指令 ROM (Instruction ROM)

`include "Data_ROM.V" // 8x8 的 資料 ROM (Data ROM)

`include "ALU_2.V" // 簡易的算術邏輯運算單元 (ALU)

module ROM_ALU (CS, Ins_Address, Data_Address, Result);

input CS;

input [2:0] Ins_Address;

input [2:0] Data_Address;

ROM_ALU.V (11)

```
output [3:0] Result;
```

```
wire [3:0] Result;
```

→ Top_module的輸出需為wire型態

```
// ----- 內部的連接線 -----
```

```
wire [1:0] opcode;
```

```
wire [3:0] data1;
```

```
wire [3:0] data2;
```

// 引用2個具有16個字組, 每個字組有3個位元的唯讀記憶體

```
Data_ROM My1_Data_ROM (CS, Ins_Address, data1, data2);
```

```
Inst_ROM My1_Inst_ROM (CS, Data_Address, opcode);
```

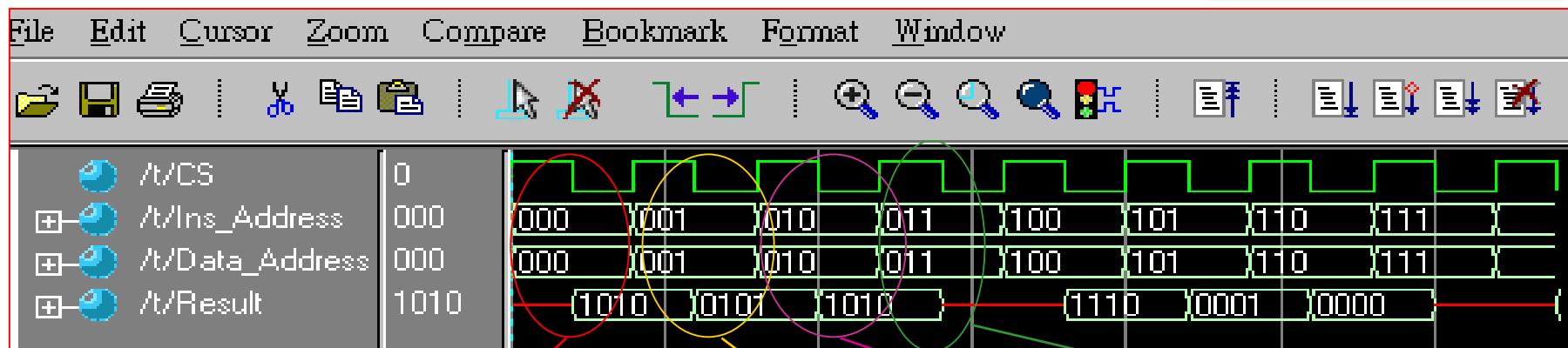
```
ALU_2 My1_ALU_2 (opcode, data1, data2, Result);
```

```
endmodule
```

in-order

Simulation of ROM_ALU.V

5_08_ROM_ALU



加法運算：
Data1+Data2
=0101+0101
=1010

減法運算：
Data1-Data2
=1010-0101
=0101

&運算：
Data1 & Data2
=1010 & 1010
=1010

無效運算碼

Data1 Data2

000→ROM_Data[0]
ROM_Data [0] = 2'b00; // 位址0的內容為 2'b00, ` Plus
ROM_Data [1] = 2'b01; // 位址1的內容為 2'b01, ` Minus
ROM_Data [2] = 2'b10; // 位址2的內容為 2'b10, ` Bin_And
ROM_Data [3] = 2'b11; // 位址3的內容為 2'b11, Non-Support
ROM_Data [4] = 2'b00; // 位址4的內容為 2'b00, ` Plus
ROM_Data [5] = 2'b01; // 位址5的內容為 2'b01, ` Minus
ROM_Data [6] = 2'b10; // 位址6的內容為 2'b10, ` Bin_And
ROM_Data [7] = 2'b11; // 位址7的內容為 2'b11, Non-Support

ROM_Data [0] = 8'b0101_0101;
ROM_Data [1] = 8'b1010_0101;
ROM_Data [2] = 8'b1010_1010;
ROM_Data [3] = 8'b0000_0000;
ROM_Data [4] = 8'b1111_1111;
ROM_Data [5] = 8'b0000_1111;
ROM_Data [6] = 8'b1111_0000;
ROM_Data [7] = 8'b0101_1010;